PROJECT SYNOPSIS

TASK MANAGER WITH PRIORITY SCHEDULING

A Cloud-Based Asynchronous Task Processing and Scheduling System

Project Type	Cloud-based Software Application
Core Concept	CRUD + Priority Scheduling + Event-Driven Processing
Messaging	Apache Kafka
Cloud Platform	Amazon Web Services (AWS)
Student	Harshit Arora
Program	B.Tech – Computer Science & Engineering
Specialization	Cloud Computing & DevOps

Synopsis Focus

Design and deployment of a scalable task management platform in which tasks are managed through CRUD operations, processed according to priority, transported asynchronously through Kafka, and deployed on AWS using cloud infrastructure concepts.

Academic Session: 2026–27

1. Introduction & Project Overview

The **Task Manager with Priority Scheduling** is a software system designed to manage tasks and process them according to their importance. Unlike a basic to-do application, the proposed system introduces a scheduling layer that assigns a priority to each task and determines the order in which pending tasks should be processed. This makes the project relevant to real-world systems where jobs are generated continuously and cannot always be executed in simple first-in-first-out order.

The current implementation provides the foundation through **CRUD operations** and priority scheduling. The planned enhancement introduces **Apache Kafka** for asynchronous event and task processing. A cloud deployment on **AWS** will provide a scalable environment for the API, workers, database, storage, monitoring and traffic management components.

2. Problem Statement

In many applications, multiple tasks may arrive at the same time, while the resources available to process them are limited. Treating every task equally can delay urgent or business-critical operations. A system is therefore required that can maintain task information, identify task priority, process tasks asynchronously, handle failures, and remain available as workload increases.

Problem addressed: How can a task management application be extended into a reliable and scalable task-processing platform where higher-priority work is handled first without losing the benefits of asynchronous processing and cloud deployment?

3. Objectives

- Develop a task management system supporting Create, Read, Update and Delete operations.
- Assign and maintain priority levels for tasks and implement priority-based scheduling.
- Use Apache Kafka to decouple task creation from task processing through producer-consumer communication.
- Introduce worker-based asynchronous processing so API requests are not tightly coupled to task execution.
- Deploy the application on AWS to demonstrate practical cloud computing concepts.
- Use database, object storage, monitoring, networking and load-balancing components.
- Design the system so it can later support retries, dead-letter queues, auto scaling and richer analytics.

4. Scope of the Project

The project covers the complete flow from task creation to scheduling and execution. The initial scope focuses on a single application domain, while the architecture is intentionally designed so that the same scheduling mechanism can later support background jobs, report generation, notifications, data processing, CI/CD jobs and other asynchronous workloads.

The project is primarily an academic prototype; production environments would require additional controls such as strong authentication, authorization, encryption, auditing, rate limiting, backup policies and domain-specific reliability requirements.

5. Proposed System Architecture

The proposed architecture follows an event-driven approach. The frontend communicates with the backend API. When a task is created or updated, the application can publish an event to Kafka rather than forcing the API to perform the entire processing operation synchronously. A consumer receives the event, places the task into the scheduling mechanism, and a worker executes the selected task.

High-Level Flow

```
User / Frontend
↓
CloudFront / Static Frontend (optional)
↓
Application Load Balancer
↓
Task Management API on EC2
↓
Kafka Producer → Kafka Topic
↓
Kafka Consumer
↓
Priority Queue / Scheduler
↓
Worker
↓
Database / Task Status
```

6. Priority Scheduling Mechanism

Every task contains a priority value. The scheduler compares pending tasks and selects the task with the highest effective priority for execution. A future enhancement can combine priority with deadline and waiting time. This is particularly useful for avoiding **starvation**, where low-priority tasks could otherwise remain pending indefinitely when high-priority work continuously arrives.

Example

Task	Priority	Expected Order
Payment Processing	10	1st
Generate Report	8	2nd
Send Notification	5	3rd
Backup / Cleanup	2	4th

7. Kafka Integration

Kafka will act as the messaging backbone between the API and task-processing services. The API behaves as a **producer** by publishing task-related events. One or more **consumers** receive these events and hand them to the scheduling and worker layer. This separation improves modularity and allows task processing to scale independently from the API.

- **Producer:** publishes events such as TASK_CREATED, TASK_UPDATED or TASK_REQUESTED.
- **Topic:** stores the stream of task events for consumers.
- **Consumer:** receives events and forwards tasks to the processing layer.
- **Worker:** executes the selected task and updates its status.
- **Retry / DLQ (future):** failed tasks can be retried and eventually moved to a dead-letter queue.

8. AWS Cloud Deployment Plan

The cloud layer is intended to demonstrate how the application can be hosted, secured, monitored and scaled using AWS. The deployment can start with a simple EC2-based setup and gradually evolve toward a more distributed architecture.

Recommended AWS Components

Component	Purpose in Project
EC2	Host backend API, Kafka/worker services or application containers.
RDS – MySQL	Persist tasks, users, statuses, priorities and execution metadata.
S3	Store attachments, reports, exports, backups or static assets.
CloudFront	Deliver a static frontend or cacheable content with low latency.
Application Load Balancer	Distribute API traffic across multiple application instances.
VPC + Subnets	Provide network isolation and organize public/private resources.
Security Groups	Control traffic between cloud components.
CloudWatch	Collect logs, metrics and operational monitoring information.
Auto Scaling	Increase or decrease application capacity based on workload.

9. Proposed AWS Architecture

```
Internet Users
↓
CloudFront → S3 (Frontend / Static Assets)
↓
Application Load Balancer
↓
EC2 Application Instances
↓
Kafka → Priority Scheduler → Worker
↓
Amazon RDS (MySQL)

CloudWatch monitors application and infrastructure activity.
```

10. Networking and Security Design

The application should be placed inside a VPC with separate public and private network areas. The public layer can contain the load-balancing entry point, while application and database resources can be placed in appropriately restricted subnets. Security groups should follow least-privilege principles—for example, the database should accept database traffic only from the application layer rather than directly from the public internet.

For a production-oriented implementation, credentials should not be hard-coded in source code. Secrets, database credentials and API keys should be managed using appropriate secret-management mechanisms, and application traffic should use HTTPS.

11. Functional Modules

Task Management Module: Create, view, update and delete tasks; maintain title, description, priority, status, deadline and timestamps.

Priority Scheduler: Maintain pending tasks and select work according to priority; future versions can support aging and deadline-aware scheduling.

Kafka Messaging Module: Publish task events and consume them asynchronously to decouple request handling from processing.

Worker Module: Execute scheduled tasks and update success, failure, retry and completion states.

Database Module: Persist task records, users, statuses, priorities and execution metadata.

Cloud Infrastructure Module: Provide compute, networking, storage, load balancing and monitoring through AWS.

Monitoring Module: Track API errors, resource utilization, task counts, processing time and failures using logs and metrics.

12. Database Design – Suggested Task Entity

Field	Description
task_id	Unique identifier for a task.
title	Task name or short description.
description	Detailed task information.
priority	Numeric priority used by scheduler.
status	Pending, Processing, Completed, Failed, etc.
deadline	Optional due time for deadline-aware scheduling.
created_at / updated_at	Lifecycle timestamps.
retry_count	Number of processing attempts.
failure_reason	Reason captured when processing fails.

13. Reliability, Scalability and Observability

- **Asynchronous processing:** Kafka prevents the API from being tightly coupled to long-running task execution.
- **Scalability:** API instances and workers can be scaled independently as demand changes.
- **Load balancing:** ALB can distribute incoming requests across healthy application instances.
- **Failure handling:** retries and a future dead-letter queue can prevent failed tasks from disappearing silently.
- **Monitoring:** CloudWatch can track API errors, CPU utilization, logs, task throughput and processing latency.
- **Storage separation:** RDS handles structured application data while S3 can hold files and generated artifacts.

14. Development & Deployment Roadmap

```
Phase 1 → CRUD Task Manager
Phase 2 → Priority Scheduling
Phase 3 → Kafka Producer / Consumer
Phase 4 → Priority Queue + Worker Processing
Phase 5 → Retry and Failure Handling
Phase 6 → Frontend Dashboard
Phase 7 → Dockerization
```

```
Phase 8 → AWS EC2 + RDS + S3 Deployment
Phase 9 → VPC + Security Groups + ALB + CloudFront
Phase 10 → CloudWatch + Auto Scaling + Performance Testing
```

15. Future Scope

The proposed architecture provides a foundation for several advanced improvements. The project can evolve from a student-level task manager into a more generalized job-processing platform.

- **Deadline-aware scheduling:** combine priority with task deadlines so urgent work receives appropriate treatment.
- **Priority aging:** gradually increase the effective priority of tasks that have waited for a long time, reducing starvation.
- **Distributed workers:** run multiple workers and use Kafka consumer groups to increase processing capacity.
- **Retry and Dead Letter Queue:** automatically retry transient failures and isolate repeatedly failing tasks.
- **Authentication and RBAC:** introduce users, roles and permissions for secure multi-user operation.
- **Task analytics:** show throughput, average waiting time, completion rate, failure rate and priority distribution.
- **Containerization:** package API, scheduler and workers with Docker for repeatable deployment.
- **Infrastructure as Code:** provision AWS resources using Terraform or AWS CloudFormation.
- **Advanced orchestration:** move workloads toward Kubernetes/EKS when the project requires larger-scale container orchestration.

16. Potential Real-World Applications

Domain	Possible Use
E-commerce	Payment, inventory and notification jobs.
DevOps / CI-CD	Build, test, scan and deployment jobs.
Reporting Systems	Generate reports according to urgency and deadlines.
Notification Platforms	Process email, SMS or push notification jobs asynchronously.
Data Processing	Schedule batch processing and background computation.

17. Expected Outcomes

- A functional task management application with CRUD operations.
- A working priority-based scheduling mechanism.
- Event-driven task processing using Kafka producer-consumer architecture.
- A worker-based asynchronous processing pipeline.
- A cloud-deployed application demonstrating AWS compute, database, storage, networking and monitoring concepts.
- A scalable architecture suitable for future enhancements such as auto scaling, retries, analytics and container orchestration.

18. Conclusion

The **Task Manager with Priority Scheduling** project combines software development with concepts from operating systems, distributed systems, messaging, cloud computing and DevOps. Starting from CRUD-based task management, the system introduces priority scheduling to make task execution more meaningful. Kafka adds asynchronous, event-driven processing, while AWS provides the infrastructure required to host, secure, monitor and scale the application.

The resulting system is more than a conventional to-do application: it is a practical prototype for understanding how modern applications separate request handling, messaging, scheduling, processing, persistence and infrastructure. The architecture also provides a clear path toward advanced features such as retries, dead-letter queues, auto scaling, observability and container orchestration.

Proposed Technology Stack

Backend: Python/Java (based on final implementation) • Database: MySQL • Messaging: Apache Kafka • Cloud: AWS • Compute: EC2 • Storage: S3 • Database Hosting: RDS • Networking: VPC, Subnets, Security Groups • Traffic: ALB • CDN: CloudFront • Monitoring: CloudWatch • Deployment Enhancement: Docker

— End of Synopsis —